Python AsyncIO

Attractive Study Notes from the Asynchronous Programming tutorial

Best for	Quick revision, interview prep, and practical Python async usage.
Source	CoreyMSchafer/AsyncIO-Code-Examples and the full tutorial transcript.

Main takeaway: asyncio helps when your program spends time waiting. It does not make CPU-heavy work magically fast. Use async for I/O, threads for blocking I/O without async support, and processes for CPU work.

1. Big Picture

What asyncio is, what it is not, and why concurrency matters.

- `asyncio` is Python's built-in library for concurrent I/O-bound code using `async` and `await`.
- It is excellent for network calls, database queries, file I/O, and other waiting-heavy workflows.
- Async does not automatically make code faster. It mainly prevents your program from sitting idle while waiting.
- `asyncio` is single-threaded and single-process by default. It relies on cooperative multitasking.

Concept	Meaning	Why it matters
Synchronous execution	One step fully finishes before the next begins.	Simple, but wastes time during waits.
Concurrency	Multiple tasks make progress over time.	Useful when tasks spend time waiting.
I/O-bound work	Time goes to external systems or the OS.	Best match for asyncio.
CPU-bound work	Time goes to computation.	Better fit for multiprocessing.

2. Core Terms You Must Know

Most asyncio confusion comes from the vocabulary, not the syntax.

Term	Definition	Example
Event loop	The engine that runs, pauses, and resumes async work.	`asyncio.run(main())`
Coroutine function	A function defined with `async def`.	`async def fetch(): ...`
Coroutine object	The awaitable created when you call an async function.	`coro = fetch()`
Task	A scheduled wrapper around a coroutine.	`asyncio.create_task(fetch())`
Future	A low-level placeholder for a result that will arrive later.	Mostly internal.
await	Pause here and give control back to the event loop.	`await asyncio.sleep(1)`

```
async def main():
    task = asyncio.create_task(fetch_data())
    result = await task

asyncio.run(main())
```

3. The Execution Model

A simple mental model makes the rest of asyncio much easier.

1. Define async work with `async def`.
2. Start the program with `asyncio.run(...)` so an event loop exists.
3. Schedule work with `asyncio.create_task(...)` when you want overlap.
4. Use `await` to pause the current coroutine and let other ready tasks run.
5. When the awaited operation finishes, the event loop resumes the paused coroutine.

Pattern	Behavior	Result
`coro = fetch(); await coro`	The coroutine starts and finishes as it is awaited.	Usually sequential behavior.
Create tasks first, await later	Both tasks are scheduled before either fully finishes.	Real concurrency.
`await task2` before `await task1`	You only guarantee not moving on until task 2 is done.	You do not force task 2 to run first.

4. Async vs Threads vs Processes

Choosing the right tool matters more than making everything async.

Tool	Best for	Typical example	Avoid when
`asyncio`	I/O-bound work with async-compatible libraries.	HTTPX, `aiofiles`, async DB drivers.	The task is CPU-heavy.
Threads	Blocking I/O when no async library exists.	`requests.get()` via `asyncio.to_thread()`.	You expect big CPU speedups.
Processes	CPU-bound heavy computation.	Image processing, numeric transforms.	The overhead is larger than the work.

- Use async libraries whenever possible instead of wrapping everything in threads.

- Threads keep the event loop responsive, but they do not fix CPU bottlenecks.
- Processes help CPU-heavy work because they bypass the main-thread limitation for computation.

5. Patterns for Running Many Tasks

The tutorial compares manual tasks, gather, and TaskGroup.

Pattern	What it does	When to prefer it
Manual `create_task`	You create tasks one by one and await them later.	When you want explicit handles to individual tasks.
`asyncio.gather(...)`	Runs many awaitables and returns results in order.	When you want collected results in one place.
`asyncio.TaskGroup()`	Structured concurrency with automatic waiting and cleanup.	When tasks should succeed or fail as a unit.

Feature	`gather(return_exceptions=True)`	`TaskGroup`
Error behavior	Continues running all tasks and returns exceptions as results.	Cancels sibling tasks when one fails.
Best use case	Batch work where partial success is acceptable.	Workflows where tasks should succeed together.
Result handling	Each position may contain either a result or an exception.	You handle the grouped failure path directly.

6. Common Pitfalls

Most bugs happen when sync thinking leaks into async code.

Mistake	What goes wrong	Fix
Using `time.sleep()` in async code	The event loop is blocked and other tasks stop making progress.	Replace it with `await asyncio.sleep(...)` or offload the blocking work.
Forgetting to `await`	Tasks may never finish before the script exits.	Await all important tasks or use a `TaskGroup`.

Mistake	What goes wrong	Fix
Assuming calling an async function starts it	You only created a coroutine object.	Schedule it with `create_task` or await it.
Using threads for CPU work	Little or no speedup for heavy computation.	Use a process pool instead.

```
# Bad
async def fetch():
    time.sleep(2)

# Good
async def fetch():
    await asyncio.sleep(2)
```

7. Real-World Optimization Workflow

The image downloader example shows how to upgrade a synchronous codebase rationally.

Step	Question	Decision in the tutorial
Profile first	Where is time actually going?	Downloads looked I/O-bound, processing looked CPU-bound.
Classify the work	Is it waiting-heavy or compute-heavy?	Downloads = I/O, image processing = CPU.
Pick the right tool	Async, threads, or processes?	Async/threads for downloads, processes for processing.
Use native async libraries	Can the sync library be replaced?	Requests was replaced by HTTPX and file writes by `aiofiles`.
Limit concurrency	Could thousands of tasks overload systems?	A semaphore limited downloads and CPU count limited workers.

Version	Download time	Processing time	Total
Synchronous baseline	~13 s	~10.5 s	~23.5 s
Async with threads for downloads	~2 s	~10.6 s	~12.7 s
Async libs + processes	~1.66 s	~3.25 s	~4.9 to 5 s

8. Repo Map and Quick Revision Sheet

Use this as the fastest way to reconnect the notes to the tutorial examples.

File	Purpose
`terms.py`	Terminology demo for sync functions, coroutines, tasks, and futures.
`example_1.py`	Synchronous baseline.
`example_2.py`	Incorrect async attempt that still behaves sequentially.
`example_3.py`	Correct concurrent version with `create_task`.
`example_4.py`	Shows that `await` does not dictate which ready task runs next.
`example_5.py`	Demonstrates blocking the event loop with sync code.
`example_6.py`	Runs blocking work in threads and CPU work in processes.
`example_7.py`	Shows `gather` and `TaskGroup`.
`real_world_example_sync_v1.py`	Original synchronous downloader + processor.
`real_world_example_async_v1.py` to `v3.py`	Incremental optimization path.

ONE-MINUTE MEMORY CHECK

1. **`async def`** defines a coroutine function.
2. Calling it creates a coroutine object.
3. **`await`** pauses the current coroutine and hands control back to the event loop.
4. **`create_task`** is what gives you overlapping async work.
5. Async for I/O, threads for blocking I/O, processes for CPU-heavy work.

Prepared as polished revision notes for the AsyncIO tutorial.